

Python Modules & Packages

♦ What is a Module?

A **module in Python** is simply a **file that contains Python code** (such as functions, variables, or classes) which can be reused in other Python programs.



Example:

A file named `'mymodule.py'` is a module.

Why Modules are Used

- Avoid writing the same code again and again
- Organize large programs into smaller parts
- Improve readability and maintainability
- Enable code sharing across different programs

♦ Types of Modules



Built-in Modules (already available in Python)

Examples:

- `math`
- `random`



User-defined Modules (created by you)

Example: `my_module.py`

2. Importing Modules

♦ Basic Import

```
import math
print(math.sqrt(16))
```

♦ Import Specific Functions

```
from math import sqrt
print(sqrt(25))
```

♦ Import with Alias

```
import math as m
print(m.factorial(5))
```

♦ Import All Functions (Not Recommended)

```
from math import *
```

3. Math Module

♦ Description

The **math module** provides mathematical functions.

♦ Common Functions

Function	Description
-----	-----
sqrt(x)	Square root
factorial(x)	Factorial
pow(x,y)	Power
ceil(x)	Round up
floor(x)	Round down

♦ Example

```
import math
```

```
print(math.sqrt(36))
print(math.factorial(5))
print(math.pow(2,3))
print(math.ceil(4.2))
print(math.floor(4.8))
```

4. Random Module

♦ Description

The **random module in Python** is a built-in module used to generate **random numbers and perform random operations** such as selecting random items, shuffling data, etc.

It provides functions to:

- Generate random numbers (integers, floats)
- Select random elements from lists
- Shuffle sequences

♦ Common Functions

Function	Description
random()	Random float (0 to 1)
randint(a,b)	Random integer
choice(list)	Random element
shuffle(list)	Shuffle list

♦ Example

```
import random
```

```
print(random.random())
print(random.randint(1,10))
```

```
lst = [1,2,3,4]
random.shuffle(lst)
print(lst)
```

```
print(random.choice(lst))
```

5. User-Defined Modules

♦ Creating a Module

Create file: `mymodule.py`

```
def add(a,b):
    return a+b
```

♦ Using the Module

```
import mymodule
print(mymodule.add(5,3))
```

6. Packages in Python

A package in Python is a collection of modules organized in directories (folders) to structure large projects in a clean and manageable way.



In simple terms:

Package = Folder → contains modules (.py files) and possibly sub-packages

Packages help to:

- Organize large code into structured form

- Avoid name conflicts between modules
- Improve readability and maintainability
- Enable hierarchical grouping (like folders inside folders)

♦ Package Structure

```
my_package/
├── __init__.py
├── module1.py
├── module2.py
├── sub_package/
│   ├── __init__.py
│   └── module3.py
```

Explanation:

- `my_package` → main package
- `module1.py`, `module2.py` → modules
- `sub_package` → nested package
- `__init__.py` → tells Python this is a package (important)

What is `__init__.py`?

- It is a special file
- Marks the directory as a **Python package**
- Can be empty or contain initialization code

Write a Python program to demonstrate the use of `__init__.py` in a package.

♦ Package Structure

```
mypackage/
├── __init__.py
├── module1.py
└── module2.py
```

◆ Step 1: Create `__init__.py`

👉 `mypackage/__init__.py`
`print("mypackage is initialized")`

`from .module1 import add`
`from .module2 import multiply`

👉 **Purpose:**

- Runs automatically when the package is imported
 - Initializes the package
 - Controls what is exposed when importing
-

◆ Step 2: Create Modules

👉 `mypackage/module1.py`
`def add(a, b):`
 `return a + b`

👉 `mypackage/module2.py`
`def multiply(a, b):`
 `return a * b`

◆ Step 3: Main Program

`import mypackage`

`print(mypackage.add(2, 3))`
`print(mypackage.multiply(4, 5))`

◆ Output

`mypackage is initialized`
`5`
`20`

◆ Explanation

- When `import mypackage` is executed:
 - The `__init__.py` file runs automatically
 - It prints the initialization message
 - It makes `add()` and `multiply()` directly accessible
-

◆ Key Points About `__init__.py`

- Marks a directory as a Python package
 - Executes automatically when the package is imported
 - Can:
 - Initialize variables
 - Import modules
 - Control package behavior
-



Conclusion (Exam Ready)

The `__init__.py` file is used to initialize a package, execute code during import, and control which modules or functions are accessible when the package is used.

Creating a Package (Step-by-Step)

Step 1: Create folder

```
my_package
```

Step 2: Add `__init__.py`

```
touch __init__.py
```

Step 3: Create modules



```
module1.py
```

```
def add(a, b):  
    return a + b
```

👉 module2.py

```
def multiply(a, b):  
    return a * b
```

♦ Importing from Packages

1. Import entire module

```
import my_package.module1  
print(my_package.module1.add(2,3))
```

2. Import specific function

```
from my_package.module1 import add  
print(add(5,6))
```

3. Using alias

```
import my_package.module1 as m  
print(m.add(10,20))
```

♦ Sub-Packages

Packages can contain other packages:

Types of Packages

1. Built-in Packages

Provided by Python

Examples:

- os
 - sys
-

2. User-defined Packages

Created by developers

3. Third-party Packages

Installed using pip

Examples:

- `numpy`
- `pandas`
- `matplotlib`

◆ Installing Packages

Difference: Module vs Package

Feature	Module	Package
Definition	Single <code>.py</code> file	Folder of modules
Size	Small	Large
Structure	Flat	Hierarchical
Example	<code>math.py</code>	<code>numpy</code>
